

快速傅里叶变换：定义和速度

快速傅里叶变换（Fast Fourier Transform, FFT）是快速计算序列的离散傅里叶变换（DFT）或其逆变换的方法。傅里叶分析可以把信号从原始域（通常是时间或空间）转换到频域，也可以完成反向转换。FFT 将 DFT 矩阵分解为稀疏因子的乘积，从而减少计算量：对于大小为 n 的数据，按 DFT 定义直接计算的复杂度为 $O(n^2)$ ，使用 FFT 则可降至 $O(n \log n)$ 。

FFT 与直接按定义计算 DFT 得到的结果相同，但计算速度更快。如果考虑实际计算中的舍入误差，许多 FFT 算法还可能比直接按定义求值更精确。

令 $X_0, \dots, X_{N-1}$ 为复数，DFT 的定义为：

$$X_k = \sum_{n=0}^{N-1} x_n e^{-i2\pi kn/N}, \quad k = 0, \dots, N-1.$$

这里共有 N 个输出 X_k ，每个输出都需要对 N 项求和。直接按定义计算时，每个输出需要 N 次复数乘法和 $N-1$ 次复数加法，因此全部 N 个输出需要 N^2 次复数乘法和 N^2-N 次复数加法，加法次数也可写成 $N(N-1)$ 。总运算量为 $O(N^2)$ ；即使省去乘以 1 等琐碎运算，也只是减少 $O(N)$ 次运算。

FFT 泛指能够在 $O(N \log N)$ 次运算内算出相同结果的方法。所有已知的 FFT 算法都具有 $O(N \log N)$ 的复杂度，但目前还没有证明表明更低的复杂度不可能实现。以常见的基 2 Cooley-Tukey 算法为例，当 N 为 2 的幂时，只需 $(N/2) \log_2 N$ 次复数乘法和 $N \log_2 N$ 次加法，就能得到相同结果。现代计算机的实际性能通常不只取决于算术运算速度，但从 $O(N^2)$ 降至 $O(N \log N)$ 的总体改进仍然很重要。

FFT 广泛应用于工程、科学和数学领域。它的基本思想早在 1805 年就被推导出来，到 1965 年得到普及。1994 年，美国数学家 Gilbert Strang 将 FFT 称为“一生中最重要的数值算法”之一；IEEE 科学与工程计算期刊也将它列入 20 世纪十大算法。